

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Техническое задание для курсовой работы по дисциплине
"Функциональное программирование"

**Система автоматизированного расчёта
минимального расхода ткани для
пошива швейного изделия**

Студент,
группы 5130201/40003

_____ Лебедев К. О.

Преподаватель

_____ Моторин Д.Е.

«_____» _____ 20__ г.

Санкт-Петербург, 2026

Содержание

1	Основание для выполнения работы	3
2	Исполнитель и соисполнители работы	3
3	Цель, задачи и исходные данные	3
3.1	Цель работы	3
3.2	Задачи, решаемые в ходе выполнения работы	3
3.3	Исходные данные	4
4	Основное содержание работы	4
4.1	Разработка проекта	4
4.2	Отчётная документация	5
5	Основные требования к выполнению работы	6
5.1	Назначение	6
5.2	Состав разрабатываемого программного комплекса	6
5.3	Функциональные требования	7
5.4	Стандартные средства и библиотеки	8
6	Этапы и сроки выполнения работы	8

1 Основание для выполнения работы

Выполнение курсового проекта по дисциплине «Функциональное программирование» в соответствии с учебным планом направления подготовки 02.03.01 «Математика и компьютерные науки» ФГАОУ ВО «СПбПУ».

2 Исполнитель и соисполнители работы

Работу выполнил студент группы 5130201/40003 Лебедев Кирилл Олегович.

3 Цель, задачи и исходные данные

3.1 Цель работы

Целью работы является разработка консольного приложения для автоматизированного расчёта оптимального расхода ткани на пошив швейного изделия с учётом типа изделия, индивидуальных мерок клиента, ширины полотна и силуэта изделия.

3.2 Задачи, решаемые в ходе выполнения работы

В процессе выполнения работы требуется решить следующие задачи:

1. Разработать типы данных для представления мерок, типов изделий, лекал и раскладов.
2. Обеспечить форматирование исходных данных формата PDF., для чего использовать сторонний парсер или разработать свой.
3. Обеспечить чтение и запись конфигураций из файлов формата JSON.
4. Реализовать алгоритм корректировки лекал, исходя из параметров клиента.
5. Реализовать алгоритм расклада лекал на ткани (2D Irregular Packing).
6. Реализовать консольный интерфейс для ввода данных и отображения результатов.
7. Реализовать систему логирования действий пользователя и ошибок выполнения.
8. Обеспечить валидацию входных данных и корректную обработку ошибок.

9. Разработать тесты с использованием библиотеки QuickCheck для модулей

3.3 Исходные данные

В качестве исходных данных для программы используются открытые материалы с сайта <https://dr-cos.info/>, представленные в формате PDF. Данные файлы содержат визуальное изображение портновских лекал. Указанное графическое представление подвергается обработке специализированным парсером, который преобразует его в полигональную форму. Каждое лекало описывается замкнутым выпуклым многоугольником, задаваемым списком точек в декартовой системе координат. Начало отсчёта координат помещается в крайнюю левую и нижнюю точку прямоугольника фигуры.

4 Основное содержание работы

4.1 Разработка проекта

Разработка проекта включает проектирование модульной архитектуры приложения на языке Haskell с использованием системы сборки **Stack**. Требуется реализовать систему типов для хранения входных и выходных данных, представление в программе лекал и раскладов. Для хранения лекал выбирается полигональное представление: каждое лекало описывается замкнутым выпуклым многоугольником, задаваемым списком точек в декартовой системе координат, как начало отсчета в которой берется самая нижняя левая точка фигуры. Ткань моделируется как прямоугольник заданной ширины и бесконечной длины.

Предлагаемый алгоритм раскладки основан на принципе NFP (No-Fit-Polygon). Краткое определение NFP выглядит следующим образом: для двух заданных многоугольников, один из которых зафиксирован, второй совершает твёрдое движение без вращения вокруг неподвижного многоугольника, скользя вдоль его контура до возвращения в исходное положение. Если на движущемся многоугольнике выбрать опорную точку, то траектория, описываемая этой точкой в процессе движения, и образует многоугольник NFP. Данный метод позволяет вычислить область, в которой два многоугольника не будут перекрываться, а также найти все возможные позиции касания между двумя фигурами. Полученные точки - вершины NFP - становятся кандидатами. Другие кандидатные позиции - точки, при размещении в которые лекало касается края или угла ткани.

Пошагово алгоритм расклада можно описать следующим образом:

1. Отсортировать лекала по убыванию площади.
2. Начать с пустого списка размещённых лекал.

3. Взять следующее лекало из списка (текущее).
4. Для каждого разрешённого поворота : выполнить поворот лекала, сгенерировать кандидатные позиции для его размещения.
5. Из всех кандидатных позиций выбрать лучшую по правилу BLF (наименьший Y , если есть кандидаты с равными Y , наименьший X).
6. Среди всех поворотов выбрать тот, который дал лучшую позицию по правилу BLF, добавить в список размещенных лекал.
7. Вернуться к шагу 3, пока не закончатся лекала.

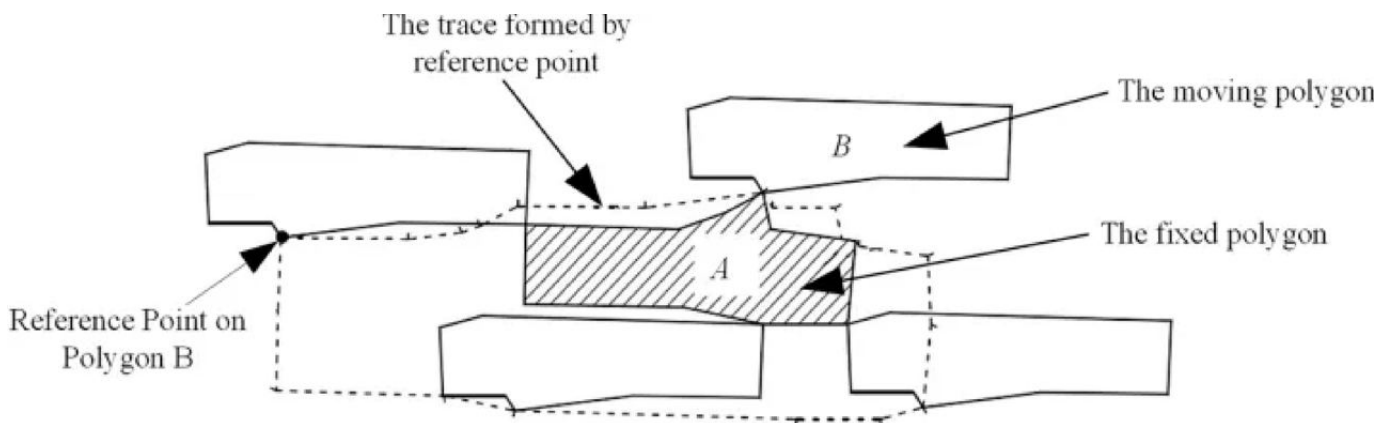


Рис. 1: Иллюстрация принципа NFP.

Реализовать программу предстоит в следующей последовательности:

- Реализация типов данных для хранения мерок, лекал, раскладов.
- Поиск стороннего парсера для форматирования исходных данных или реализация собственного.
- Реализация алгоритма расклада на ткани, относящегося к задаче "2D Irregular Packing"
- Реализовать консольный интерфейс для взаимодействия с пользователем.
- Реализовать систему логирования и обработки ошибок.

4.2 Отчётная документация

Отчётная документация содержит:

- Отчет о выполнении курсовой работы.
- Репозиторий реализованной программы.

5 Основные требования к выполнению работы

5.1 Назначение

Разрабатываемый программный комплекс предназначен для швей, модельеров, сотрудников ателье и студентов швейных специальностей для быстрого и точного расчёта необходимого объёма ткани перед раскроем, что позволяет минимизировать отходы и затраты материалов.

5.2 Состав разрабатываемого программного комплекса

Программный комплекс реализуется в виде модульной архитектуры на языке Haskell. Состав модулей:

- **App.Main** – точка входа в приложение, запуск консольного интерфейса с дефолтной конфигурацией.
- **Модули пользовательского интерфейса:**
 - Input.CLI – консольное меню, ввод пути к PDF-файлу, мерок клиента, типа изделия, силуэта, ширины ткани, а также пути сохранения результата.
 - Input.FileLoader – чтение PDF-файлов и JSON-конфигураций с диска, проверка существования и доступности файлов.
- **Модули парсинга:**
 - Parse.PDF – извлечение полигонального представления лекал из PDF-документа, нормализация координат относительно нижней-левой точки фигуры.
 - Parse.JSON – сериализация и десериализация конфигураций приложения и мерок клиента в формате JSON.
- **Модуль корректировки лекал:**
 - Adjust.Pattern – масштабирование лекал по индивидуальным меркам клиента, применение конструкционных припусков в зависимости от силуэта изделия, валидация мерок.
- **Модули расклада:**
 - Layout.NFP – вычисление No-Fit Polygon и проверка пересечения полигонов.
 - Layout.Packing – алгоритм двумерной упаковки лекал на ткани с использованием эвристики Bottom-Left Fill, поддержка поворотов на углы, кратные 90°.

- **Модули вывода:**

- `Output.Console` – вывод результатов расклада, отчётов и сообщений об ошибках в терминал.
- `Output.FileWriter` – сохранение расклада, отчётов и конфигураций в файлы.

- **Модули конфигурации и типов данных:**

- `Config.App` – определение типов `AppConfig` и `LogConfig`, дефолтные значения параметров приложения.
- `Types.Geometry` – базовые геометрические типы (`Point`, `Polygon`, `Angle`, `BoundingBox`) и операции над ними (площадь, ограничивающий прямоугольник, перенос, поворот).
- `Types.Garment` – типы предметной области: `Measurements`, `GarmentType`, `Silhouette`, `Pattern`.
- `Types.Layout` – типы для представления раскладки: `Fabric`, `PlacedPattern`, `Layout`, `AppReport`.

- **Модуль логирования:**

- `Log` – запись событий в файл с временными метками, инициализация системы логирования, уровни `Info`, `Warning`, `Error`.

- **Модуль обработки ошибок:**

- `Error` – иерархия типов ошибок (`ParseErr`, `IOErr`, `ValidationErr`, `LayoutErr`) и функции их обработки, обеспечивающие корректное завершение программы без аварийного выхода.

5.3 Функциональные требования

1. **Чтение и парсинг исходных данных.** Программа должна читать файлы формата PDF с сайта <https://dr-cos.info/> и преобразовывать визуальное представление лекал в замкнутые выпуклые многоугольники, задаваемые списком точек в декартовой системе координат. Начало отсчёта — крайняя левая нижняя точка фигуры.
2. **Корректировка лекал.** На основе введённых мерок клиента, типа изделия и силуэта программа должна масштабировать лекала с учётом конструктивных припусков.
3. **Расчёт расклада лекал.** Реализация алгоритма двумерной упаковки (2D Irregular Packing) на основе метода No-Fit Polygon (NFP) и эвристики

Bottom-Left Fill (BLF) с поддержкой поворота лекал на углы, кратные 90°. Результат — размещение всех лекал на ткани заданной ширины с минимальной длиной.

4. **Консольный интерфейс.** Текстовое меню для ввода данных, загрузки/сохранения конфигураций, запуска расчёта и просмотра результатов.
5. **Работа с файлами.** Чтение и запись конфигураций в формате JSON; сохранение отчёта о результатах расчёта в текстовый файл.
6. **Логирование.** Запись всех действий пользователя и ошибок в файл `app.log` с временными метками.
7. **Обработка ошибок.** Корректная реакция на повреждённые или отсутствующие файлы, некорректный ввод пользователя (отрицательные мерки, нечисловые значения), несовместимость параметров. Программа не должна аварийно завершаться.

5.4 Стандартные средства и библиотеки

При написании реализации программы приоритет должен отдаваться стандартным средствам языка и библиотекам.

6 Этапы и сроки выполнения работы

Таблица 1: Этапы разработки программы

№	Содержание этапа	Плановая дата
1	Написание и защита ТЗ, утверждение темы	[24.04]
2	Разработка диаграммы типов и модулей, выбор библиотек	[31.04]
3	Реализация программы	[14.05]
4	Написание тестов, составление отчёта	[21.05]

№	Этап	10.апр	24.апр	31.апр	14 мая	21 мая
1	Написание и защита ТЗ, утверждение темы					
2	Разработка диаграммы типов и модулей, выбор библиотек					
3	Реализация программы					
4	Написание тестов, составление отчёта					

Рис. 2: Диаграмма Ганта.